Script Presentasi Final — Kelompok 1

Oct 6, 2026 · @Atika

Pemateri	Slide	Materi	Durasi
Atika	01–04	Pembukaan, tim, background, problem statement	3,5 menit
Husnu	05–06	Objectives, functional requirements, arsitektur	3,5 menit
Eka	07–08	Pemetaan fitur ke struktur data	2,5 menit
Febri Anggoro	09–10	Heap & Priority Queue	2,5 menit
Rigan	11–12	Hash Table	2,5 menit
Rendi	13–14	Trie	2,5 menit
Febri Andri	15–16	AVL Tree	2,5 menit
M Febri	17–18	Dynamic Programming	3,5 menit
Daffa	19–20	Complexity, testing, eksperimen	3,5 menit
Atika	21–22	Timeline, kesimpulan, referensi	1,5 menit
Total			**28 menit**

Atika · Slide 01–04

Pembukaan, tim, background, problem statement · 3,5 menit

Slide 01 · Cover

Selamat pagi, Pak Lawrence, dan teman-teman. Kami Kelompok 1. Hari ini kami mempresentasikan rancangan Smart Delivery and Order Optimization System, yaitu sistem yang membantu platform food delivery mengelola order dalam jumlah besar, menentukan prioritas pengiriman, mencari data dengan cepat, dan menentukan rute antar yang optimal.

Slide 02 · Kelompok 1

Kelompok kami terdiri dari sembilan orang. Saya Atika, koordinator tim dan penanggung jawab laporan. Husnu dan Eka memegang Heap dan Priority Queue, Febri Anggoro memegang Hash Table, Rigan memegang Trie, dan Rendi memegang AVL Tree. M Febri dan Febri Andri mengkaji Red-Black Tree dan B-Tree sebagai pembanding yang mendasari pemilihan AVL Tree. Daffa memegang Dynamic Programming. Setiap anggota akan membawakan satu bagian presentasi hari ini.

Slide 03 · Business background

Bayangkan aplikasi food delivery pada jam makan siang. Dalam hitungan menit masuk ribuan order, dan setiap order harus diputuskan: mana yang diproses duluan, dari restoran mana, dan kurir mengantar dengan urutan apa. [Sisipkan 1–2 angka industri bersumber, atau lewati kalimat ini jika placeholder di slide sudah dihapus.]

Untuk merancang sistem, kami memakai asumsi skala di sebelah kanan: seratus ribu order aktif per hari, sepuluh ribu restoran, seratus lima puluh ribu item menu, dan lima sampai lima belas titik antar per kurir. Angka ini asumsi desain kami, bukan data industri, dan menjadi ukuran data terbesar pada eksperimen nanti. Pada skala ini, menyimpan semua order dalam satu list biasa akan semakin lambat setiap kali data bertambah. Karena itu setiap kebutuhan kami petakan ke struktur data yang paling sesuai.

Slide 04 · Problem statement

Dari situ kami merumuskan lima masalah. Pertama, order Urgent harus selalu diproses sebelum High dan Normal; ini dijawab Heap dan Priority Queue. Kedua, order harus ditemukan dari nomornya dalam rata-rata O(1); ini tugas Hash Table. Ketiga, saran restoran harus muncul dari potongan kata yang diketik; ini dijawab Trie. Keempat, peringkat restoran harus tetap terurut walaupun rating terus berubah; ini tugas AVL Tree. Kelima, kurir harus mendapat rute tercepat tanpa mencoba semua n faktorial urutan; ini dijawab dynamic programming. Jadi setiap masalah dipasangkan dengan tepat satu struktur data.

Selanjutnya, Husnu akan menjelaskan tujuan dan fitur yang akan kami bangun.

Husnu · Slide 05–06

Objectives, functional requirements, arsitektur · 3,5 menit

Slide 05 · Tujuan dan fitur (1,5 menit)

Terima kasih, Atika. Berdasarkan lima masalah tadi, kami menetapkan enam tujuan yang terlihat di kiri. Lima tujuan pertama adalah merancang masing-masing struktur data dan algoritma: Priority Queue berbasis Min-Heap, Hash Table dengan separate chaining, Trie untuk prefix search, AVL Tree dengan empat jenis rotasi, dan DP Held-Karp. Tujuan keenam adalah menyusun rencana pengujian dan eksperimen.

Di kanan adalah fitur yang akan kami bangun di UAS: tujuh belas functional requirement dalam enam modul, yaitu Order Management, Priority, Search, Ranking, Optimization, dan Performance Lab untuk eksperimen seribu sampai seratus ribu data. Fitur kami bagi menjadi wajib dan opsional, supaya rencana UAS tetap realistis.

Slide 06 · Arsitektur sistem (2 menit)

Semua fitur itu kami susun dalam empat lapisan. Lapisan pertama adalah tampilan Streamlit dengan lima halaman. Lapisan kedua adalah service layer yang berisi logika bisnis; setiap halaman hanya berbicara dengan service-nya sendiri. Lapisan ketiga, yang kami sorot, adalah inti mata kuliah ini: lima modul struktur data dan algoritma yang kami implementasikan sendiri tanpa library siap pakai. Lapisan keempat adalah data sintetis yang dibangkitkan dengan seed tetap, sehingga eksperimen bisa diulang dengan hasil yang sama.

Contoh alurnya: ketika order baru masuk, OrderService menyimpannya ke Hash Table dengan kunci order_id, sekaligus memasukkannya ke Priority Queue. Saat dispatch, order paling urgent diambil dari Priority Queue, detailnya diambil dari Hash Table, lalu beberapa order untuk satu kurir dikirim ke modul dynamic programming untuk dihitung rutenya. Pemisahan lapisan ini membuat setiap struktur data bisa diuji dan diukur performanya tanpa bergantung pada tampilan.

Selanjutnya, Eka akan menjelaskan bagaimana setiap fitur dipetakan ke struktur data.

Eka · Slide 07–08

Pemetaan fitur ke struktur data · 2,5 menit

Slide 07 · Mapping fitur bisnis (1,5 menit)

Naskah ini mengasumsikan slide 07 sudah diperbaiki menjadi Min-Heap dan tabulation.

Terima kasih, Husnu. Tabel ini memetakan setiap fitur bisnis ke struktur data beserta alasan utamanya. Priority Order Management memakai Min-Heap dan Priority Queue, karena order paling urgent selalu berada di root; insert dan extract O(log N), peek O(1). Fast Order Search memakai Hash Table dengan chaining, karena order dicari langsung dari ID-nya tanpa memeriksa satu per satu; rata-rata O(1). Restaurant and Menu Search memakai Trie, karena setiap prefix punya satu jalur sehingga cocok untuk autocomplete; waktunya O(L tambah K), dengan L panjang prefix dan K jumlah hasil.

Restaurant Ranking memakai AVL Tree yang selalu seimbang, dan penelusuran in-order-nya langsung menghasilkan urutan ranking. Delivery Optimization memakai dynamic programming bitmask, karena sub-masalah rute muncul berulang kali; hasilnya dicatat di tabel, sehingga kompleksitas turun dari O(n faktorial) menjadi O(n kuadrat kali dua pangkat n). Baris terakhir, Red-Black Tree dan B-Tree, kami kaji sebagai pembanding yang mendasari keputusan memilih AVL; B-Tree lebih cocok untuk indeks data di disk.

Slide 08 · Kunci dan alternatif (1 menit)

Slide ini melengkapi tabel sebelumnya dengan kunci yang disimpan setiap struktur dan alternatif yang kami tolak. Misalnya, Priority Queue menyimpan pasangan level, waktu, dan order_id, sedangkan AVL Tree menyimpan pasangan rating dan restaurant_id. Prinsip pemilihannya: kami memilih struktur yang operasi utamanya paling sering dipakai fitur tersebut. Alternatif di kolom terakhir tidak kami buang, justru kami pakai sebagai pembanding di eksperimen performa.

Kita mulai dari struktur data pertama. Febri Anggoro akan menjelaskan Heap dan Priority Queue.

Febri Anggoro · Slide 09–10

Heap & Priority Queue · 2,5 menit

Slide 09 · Order paling urgent selalu di puncak (1,5 menit)

Terima kasih, Eka. Kebutuhan utama fitur prioritas adalah selalu mengambil order paling urgent, bukan mengurutkan semua order. Untuk itu kami memakai Min-Heap.

Kuncinya adalah pasangan level dan waktu masuk. Level 1 berarti Urgent, 2 High, dan 3 Normal. Karena angka terkecil berarti paling urgent, Min-Heap adalah pilihan yang tepat. Jika levelnya sama, waktu masuk yang lebih awal menang, sehingga berlaku first in, first out.

Di diagram, root berwarna oranye adalah order Urgent yang masuk pukul 08.10. Aturannya: setiap parent selalu lebih kecil atau sama dengan anak-anaknya. Heap ini disimpan sebagai array biasa seperti baris di bawahnya, sehingga hemat memori.

Keunggulannya, insert dan extract hanya O(log n): dengan satu juta order, tinggi heap hanya sekitar dua puluh tingkat. Keterbatasannya, mencari order tertentu di dalam heap butuh O(n), karena itu pencarian kami serahkan ke Hash Table.

Slide 10 · Pseudocode (1 menit)

Saat insert, order baru ditaruh di akhir array, lalu sift_up menukarnya ke atas selama ia lebih kecil dari parent-nya. Saat extract_min, root diambil, elemen terakhir dipindah ke posisi root, lalu sift_down menukarnya ke bawah dengan anak terkecil sampai urutannya benar lagi. Jumlah pertukaran paling banyak setinggi pohon, sehingga keduanya O(log n), sedangkan peek cukup O(1).

Berikutnya, Rigan akan menjelaskan Hash Table untuk pencarian order.

Rigan · Slide 11–12

Hash Table · 2,5 menit

Slide 11 · Order ditemukan langsung dari nomornya (1,5 menit)

Terima kasih, Febri. Customer service dan kurir mencari order dari nomornya, yaitu order_id. Karena ini pencarian kunci yang persis, Hash Table adalah pilihan yang paling tepat.

order_id diubah oleh fungsi hash, yaitu rolling hash polinomial, menjadi indeks bucket. Ukuran tabel kami buat bilangan prima agar sebaran indeks lebih merata.

Collision terjadi ketika dua kunci berbeda menghasilkan indeks yang sama. Di diagram, ORD-0007 dan ORD-0042 sama-sama jatuh ke bucket 3. Kami menanganinya dengan separate chaining: keduanya disimpan berderet dalam satu chain.

Kenapa bukan open addressing? Di sistem kami, order sering dihapus karena dibatalkan atau selesai. Pada open addressing, penghapusan membutuhkan penanda tombstone yang lama-lama memperlambat pencarian. Saat data membesar, tabel kami perbesar dua kali lipat ketika load factor melewati 0,75, sehingga rata-rata tetap O(1).

Slide 12 · Pseudocode (1 menit)

Put mencari dulu apakah kunci sudah ada di chain. Kalau ada, nilainya diganti; kalau belum, pasangan baru ditambahkan, lalu load factor diperiksa. Get cukup menelusuri satu chain. Kasus terburuknya O(n), yaitu jika semua kunci jatuh di bucket yang sama, tetapi rehash menjaga chain tetap pendek. Biaya rehash O(n) jarang terjadi, sehingga rata-ratanya tetap O(1).

Selanjutnya, Rendi akan menjelaskan Trie untuk pencarian restoran dan menu.

Rendi · Slide 13–14

Trie · 2,5 menit

Slide 13 · Saran restoran muncul saat pengguna mengetik (1,5 menit)

Terima kasih, Rigan. Saat mencari restoran, pengguna tidak mengetik nama lengkap, melainkan potongan awalnya. Untuk kebutuhan prefix search seperti ini kami memakai Trie.

Di Trie, setiap huruf adalah satu node, dan kata dengan awalan sama berbagi jalur yang sama. Lihat diagram: ketika pengguna mengetik "ay", kita cukup turun dua langkah, ke node a lalu node y yang berwarna oranye. Semua kata di bawah node itu, yaitu ayam bakar dan ayam geprek, langsung menjadi saran.

Waktunya O(m tambah k): m adalah panjang ketikan dan k jumlah saran. Artinya kecepatannya tidak bergantung pada jumlah restoran. Kenapa bukan Hash Table? Hash Table hanya bisa mencari kata yang utuh, tidak bisa mencari berdasarkan awalan. Keterbatasan Trie adalah memorinya boros karena setiap huruf menjadi node; karena itu hasil saran kami batasi sepuluh.

Slide 14 · Pseudocode (1 menit)

Starts_with bekerja dalam dua tahap. Pertama, telusuri node per huruf dari prefix; jika ada huruf yang tidak ditemukan, langsung kembalikan daftar kosong. Kedua, dari node terakhir, kumpulkan semua kata dengan penelusuran DFS berurutan alfabet sampai batas tercapai. Semua teks dinormalisasi ke huruf kecil, jadi BAKSO dan bakso memberi hasil yang sama.

Berikutnya, Febri Andri akan menjelaskan AVL Tree untuk ranking restoran.

Febri Andri · Slide 15–16

AVL Tree · 2,5 menit

Slide 15 · Ranking tetap seimbang lewat rotasi (1,5 menit)

Terima kasih, Rendi. Fitur ranking membutuhkan data yang selalu terurut, bisa mengambil top-k dan memfilter rating, serta tetap cepat walaupun rating terus berubah. Untuk itu kami memakai AVL Tree. Kuncinya pasangan rating dan restaurant_id; ID diperlukan karena banyak restoran memiliki rating yang sama.

AVL selalu seimbang: selisih tinggi subtree kiri dan kanan di setiap node paling banyak satu. Jika dilanggar, pohon diperbaiki dengan rotasi. Contohnya di diagram: jika rating 4,2, lalu 4,5, lalu 4,8 dimasukkan berurutan, pohon miring ke kanan dan balance factor node 4,2 menjadi minus dua. Ini kasus RR, diselesaikan dengan satu rotasi kiri sehingga 4,5 naik menjadi root. Input yang turun memicu kasus LL, dan urutan zig-zag memicu LR atau RL.

Kami memilih AVL dibanding Red-Black Tree karena AVL lebih ketat seimbang sehingga pencarian lebih cepat, dan ranking lebih sering dibaca daripada diubah. B-Tree lebih cocok untuk indeks data yang tersimpan di disk.

Slide 16 · Pseudocode (1 menit)

Insert menyisipkan kunci seperti BST biasa, lalu menghitung balance factor. Empat kondisi di bawahnya menentukan rotasi: LL dengan rotasi kanan, RR dengan rotasi kiri, LR dengan rotasi kiri lalu kanan, dan RL sebaliknya. Satu rotasi hanya O(1), dan karena pohon selalu seimbang, insert, delete, dan search tetap O(log n). Update rating dilakukan dengan delete lalu insert ulang, karena kuncinya ikut berubah.

Selanjutnya, M Febri akan menjelaskan rencana dynamic programming untuk rute pengiriman.

M Febri · Slide 17–18

Dynamic Programming · 3,5 menit

Slide 17 · Rute kurir (2 menit)

Terima kasih, Febri. Masalah yang kami optimalkan adalah rute kurir. Seorang kurir membawa pesanan dari satu restoran dan harus mengantar ke n customer. Kami mencari urutan antar dengan total waktu paling kecil, dan kurir tidak perlu kembali ke restoran.

State kami adalah dp[mask][j], yaitu waktu minimum untuk mengunjungi semua titik di dalam mask dan berakhir di titik j. Mask adalah bilangan biner: bit ke-i bernilai satu jika titik i sudah dikunjungi. Base case-nya: untuk mask yang hanya berisi restoran, nilainya nol, karena kurir mulai dari sana. Jawaban akhirnya adalah nilai terkecil dp ketika semua titik sudah dikunjungi.

Di kanan ada contoh kecil: satu restoran dan tiga customer, dengan waktu tempuh dalam menit. Ada enam kemungkinan urutan, dan rute terbaik adalah R ke C1, ke C2, lalu ke C3, dengan total empat tambah tiga tambah dua, yaitu sembilan menit.

Slide 18 · Recurrence dan perbandingan (1,5 menit)

Recurrence-nya seperti di layar: untuk menambahkan customer k yang belum dikunjungi, kita ambil biaya terbaik yang berakhir di j, lalu tambahkan waktu dari j ke k. Hasil kecil ini dicatat di tabel dan dipakai ulang, tidak dihitung ulang.

Brute force mencoba semua urutan dengan kompleksitas O(n kali n faktorial): untuk sepuluh customer, sekitar tiga puluh enam juta langkah. Held-Karp O(n kuadrat kali dua pangkat n): untuk sepuluh customer, sekitar seratus ribu langkah.

Kami memilih tabulation, karena mask bisa diisi berurutan dari kecil ke besar tanpa rekursi, sehingga tidak ada risiko batas rekursi Python. Keterbatasannya, Held-Karp tetap eksponensial, sehingga realistis untuk satu batch kurir sekitar lima belas titik, bukan seluruh kota.

Selanjutnya, Daffa akan menjelaskan rencana analisis kompleksitas dan pengujian.

Daffa · Slide 19–20

Complexity, testing, eksperimen · 3,5 menit

Slide 19 · Big-O teoritis (2 menit)

Terima kasih, M Febri. Tabel ini merangkum kompleksitas teoritis semua operasi inti. Huruf n berarti jumlah data, m panjang kata atau prefix, dan k jumlah hasil.

Operasi Heap dan AVL Tree berada di O(log n). Hash Table rata-rata O(1), dengan kasus terburuk O(n) jika semua kunci bertabrakan. Trie bergantung pada panjang ketikan, bukan jumlah data. Held-Karp O(n kuadrat kali dua pangkat n), jauh lebih kecil dibanding brute force O(n kali n faktorial).

Lalu apa yang terjadi jika dataset menjadi satu juta? Untuk Heap dan AVL, log basis dua dari satu juta sekitar dua puluh, jadi setiap operasi hanya bertambah beberapa langkah. Hash Table tetap rata-rata O(1) selama tabel diperbesar saat load factor melewati 0,75. Sedangkan DP tidak bergantung pada jumlah order, melainkan jumlah titik per kurir. Dari sisi memori, Trie paling boros, sebanding dengan total huruf yang disimpan; ini trade-off yang kami terima demi pencarian prefix yang cepat.

Slide 20 · Testing dan eksperimen (1,5 menit)

Untuk membuktikan rancangan ini, kami menyiapkan delapan belas test case: empat untuk Heap, empat untuk Hash Table, tiga untuk Trie, empat untuk AVL Tree yang masing-masing menguji rotasi LL, RR, LR, dan RL, dua untuk DP termasuk memastikan hasilnya sama dengan brute force, dan satu test integrasi.

Ada juga lima eksperimen performa. Setiap struktur data dibandingkan dengan pendekatan naif, misalnya AVL dengan BST tanpa balancing yang kami uji dengan input terurut. Ukuran data seribu, sepuluh ribu, lima puluh ribu, dan seratus ribu. Kami mengukur waktu, jumlah operasi, dan memori, lalu menampilkannya sebagai grafik di Streamlit.

Untuk menutup, Atika akan menyampaikan timeline dan kesimpulan.

Atika · Slide 21–22

Timeline, kesimpulan, referensi · 1,5 menit

Slide 21 · Rencana enam minggu menuju UAS

Terima kasih, Daffa. Ini rencana enam minggu kami menuju UAS. Minggu pertama menyiapkan repository, generator data, dan service layer. Minggu kedua, keempat struktur data diimplementasikan secara paralel beserta unit test-nya. Minggu ketiga untuk dynamic programming dan integrasi. Minggu keempat untuk halaman Streamlit. Minggu kelima menjalankan seluruh test dan eksperimen. Minggu keenam menyusun laporan dan berlatih presentasi.

Sebagai kesimpulan, setiap fitur dalam rancangan kami sudah memiliki struktur data, justifikasi teoritis, pseudocode, rencana pengujian, dan penanggung jawabnya. Kami yakin rancangan ini realistis dan siap masuk tahap implementasi.

Terima kasih atas perhatiannya. Kami persilakan Bapak untuk memulai sesi tanya jawab.